

Large-Scale Ontology Storage and Query Using Graph Database-Oriented Approach: The Case of Freebase

Mahmoud Elbattah
College of Engineering &
Informatics,
National University of
Ireland
m.elbattah1@nuigalway.ie

Mohamed Roushdy
Faculty of Computer and
Information Sciences
Ain Shams University, Cairo,
Egypt
mroushdy@cis.asu.edu.eg

Mostafa Aref
Faculty of Computer and
Information Sciences
Ain Shams University, Cairo,
Egypt
aref_99@yahoo.com

Abdel-Badeeh M. Salem
Faculty of Computer and
Information Sciences
Ain Shams University, Cairo,
Egypt
absalem@cis.asu.edu.eg

Abstract— Ontology has been increasingly recognised as an instrumental artifact to help make sense of large amounts of data. However, the challenges of Big Data significantly overburden the process of ontology storage and query particularly. In this respect, the paper aims to convey considerations in relation to improving the practice of storing or querying large-scale ontologies. Initially, a systematic literature review is conducted with the aim of thoroughly inspecting the state-of-the-art in literature. Subsequently, a graph database-oriented approach is proposed, considering ontology as a large graph. The approach endeavours to address the limitations encountered within traditional relational models. Furthermore, scalability and query efficiency of the approach are verified based on empirical experiments using a subset of Freebase data. The Freebase subset is utilised to build a large-scale ontology graph composed of more than 500K nodes, and 2M edges.

Keywords—Ontology; Graph Database; Freebase

I. INTRODUCTION

Ontology can be concisely defined as an explicit specification of conceptualization [30]. Largely recognised, ontology can provide higher potentials for understanding of how data can be used. Therefore, the role of ontology gained a considerable significance, especially in the wake of Big Data. However, we argue that the commonly used approaches and tools for ontology storage or query still lag behind towards enabling the usability of large-scale ontology models. Specifically, an ontology storage model should endorse the following points:

1. Model Understand-ability: How does the storage model conceptually structure an ontology model?
2. Model Flexibility: How flexibly can the storage model adapt to ontology changes?
3. Model Query-ability: How effectively ontologies can be queried?
4. Model Scalability: How scalable is the storage model?

According to [10], there are two main approaches for storing ontologies based on databases as follows: i) single table approach, and ii) dual schemes approach. In the single table approach, such as [11] [21], the description of classes, properties and their instances are all stored in a single table called vertical table [12]. The schema of the vertical table has three columns: subject, predicate and object, which respectively represent instance identifier, instance property and instance value. While the second approach, the dual scheme, separately stores ontologies and instance data in two different structures, called ontology and data, respectively [13].

In contrast to the two above-mentioned approaches, we propose a graph database-oriented approach that stores an ontology model as a single large graph. Furthermore, graph-driven query operations are adopted in order to realise complex ontology query scenarios. The paper is organised as follows: First, a systematic literature review on storage models of ontology. Second, the proposed approach is delineated. Eventually, an empirical case study is presented using Freebase [32] dataset. Freebase is considered as one of the largest sources in the Linked Open Data (LOD) cloud, which can provide a rich source of ontologies.

II. REVIEW PROCESS

The preliminary stage of the study adhered to a systematic literature review using methods informed by [9]. The review sought to comprehensively include state-of-the-art approaches and methods adopted for storing and querying ontology. Moreover, the review explored the appropriateness of reviewed approaches in context of large-scale ontologies.

A. Search Strategy

Initially, we posed the four investigative questions presented in Table I in order to focus the review process. Literally, the search was conducted using two principal keywords as follows: i) storing ontology, and ii) querying ontology. The search process targeted four major digital libraries including: i) IEEE Xplore, ii) ACM Digital Library, iii) ScienceDirect – Elsevier, and iv) SpringerLink.

TABLE I. Investigative Questions and Motivations

Question	Motivation
Q1: What types of ontology storage models exist?	Review state-of-the-art techniques for ontology storage.
Q2: What are the ontology query capabilities provided by those models?	Investigate the query operations and capabilities provided by those models identified by Q1.
Q3: What are the common standardised query languages for ontology?	Inspect the well-established query languages for ontology.
Q4: How appropriate are the current ontology stores for addressing large-scale ontologies?	Inspect the suitability of the reviewed ontology stores for storing/querying large-scale ontologies.

B. Stages of Review Process

The review process was accomplished through four stages. Stage 1 included searching digital libraries for potential relevant studies using the aforementioned search keywords. Stage 2 excluded irrelevant studies based on titles. Stage 3 excluded irrelevant studies based on abstracts, including studies that directly addressed storing or querying of ontologies. Stage 4 involved inclusion and critical appraisal of significant studies. The stages of the review process are sketched in Figure 1, where the number of included papers is identified at each stage.

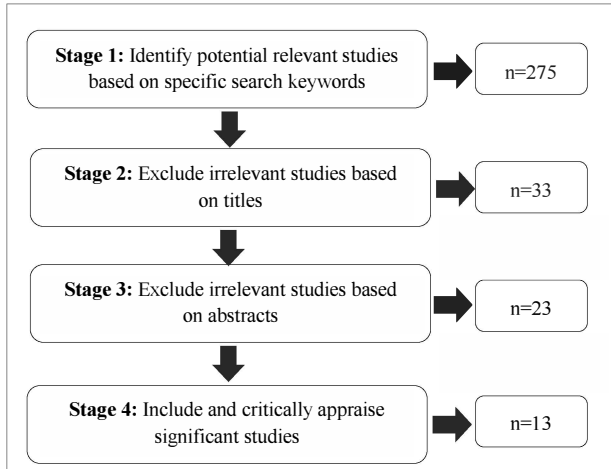


Fig. 1 Stages of the review process

C. Synthesis of Findings

Based on the reviewed studies, numerous studies adopted the relational model for ontology storage or query, such as studies [14-22]. For instance, study [19] proposed a relational database layout to store an OWL knowledge base by considering the hierarchical relations. Moreover, other recent studies such as [25] investigated the usability of object-oriented databases for storing OWL ontologies. However, we argue that the relational model can have the following limitations in this context:

- Storing ontologies in a relational database will essentially require a mapping overhead, since that the relational model defines its knowledge in a way

different from ontology, which is based mainly on tables and relations.

- Relational schemas are fixed and rigorous, while ontologies are dynamically extensible.
- The relational databases offer limited reasoning capability for ontology, acknowledged by [23].
- Further, ontology stored on a relational database may suffer from scalability issues, addressed by [24].

In concrete, the review evidently revealed that the literature seldom laid emphasis on embracing graph-driven approaches for storing or querying ontologies. Specifically, only few studies such as [26] and [27] used graph databases.

III. PROPOSED APPROACH: GRAPH DATABASE AS ONTOLOGY STORE

A. What is a Graph Database Model?

From a database point of view, a data model is a collection of conceptual tools for describing the real-world entities, and the relationships among these entities to be modelled in the database [2]. With regard to graph databases, the data model is principally based on graph structures or generalizations of them [1]. Therefore, data structures are simply modelled as nodes and edges, which can make it possible to manipulate data using explicit graph-driven operations.

B. Why a Graph Database?

In essence, ontologies can be conceived as graphs, which store metadata about data with regard to topology, relations and properties. According to [3], graphs have the advantage of being able to keep all the information about an entity in a single node and showing related information by arcs connected to it. Therefore, graph models can intuitively be the best fit to depict an ontology model. In view of that, it is expected that graph databases can offer promising potentials for the persistence and querying of ontologies in terms of:

- Supporting a flexible schema-less data model that enables to model and manage rich domains in an intuitive way.
- Being optimized for graph traversal of highly interconnected data, which can answer queries requiring extensive navigation around a graph.
- Being a member of the NoSQL family, they have the advantage to scale horizontally in order to dynamically adapt to the addition of new servers [4].

C. Selection of the Graph Database Platform

The Neo4j graph database was selected as an exemplar platform towards realising the study objectives. Neo4j is an open-source NoSQL graph database implemented in Java and Scala [5]. Neo4j implements the property graph model down to the storage level. More importantly, Neo4j provides a SQL-like query language called “Cypher”, which can be effectively used for expressing ontology query operations. Cypher is built on the basic concepts and clauses of SQL, but has a lot of additional graph-specific functionality to make it

simple to work with a rich graph model without being too verbose [6].

Furthermore, the selection of Neo4j was decided based on survey studies such as [7] and [8], which conducted formal reviews of graph databases. According to [7], DEX and Neo4j graph databases had the most efficient performance based on HPC scalable graph analysis benchmarking. While study [8] reviewed the strengths of Neo4j against other graph database platforms as demonstrated in Table II.

Table II. Review of Graph Databases with respect to Graph Modeling Capabilities

Graph Database	Graphs Nodes			Nodes			Edges		
	Simple graphs	Hypergraphs	Nested Graphs	Attributed Graphs	Node labelled	Node attribution	Directed	Edge labelled	Edge attribution
Allegro Graph	✓				✓		✓	✓	
DEX				✓	✓	✓	✓	✓	✓
Filament	✓				✓		✓	✓	
G-Store	✓				✓		✓	✓	
Hyper GraphDB		✓			✓	✓	✓	✓	✓
InfiniteGraph				✓	✓	✓	✓	✓	✓
Neo4j				✓	✓	✓	✓	✓	✓
Sones		✓		✓	✓	✓	✓	✓	✓
vertexDB	✓				✓		✓	✓	

IV. EMPIRICAL PERSPECTIVE: ONTOLOGIZING FREEBASE

In this section, we present an empirical case study to verify the validity of our approach, and investigate its usefulness regarding large-scale ontologies.

A. Freebase Dataset

Freebase is a huge, collaboratively database of cross-linked data developed by Metaweb Technologies [28]. Freebase includes a great diversity of structured data imported from different sources such as Wikipedia, MusicBrainz and WordNet [2]. Freebase includes about 50 million topics. As mentioned in Section 1, Freebase data was utilised for constructing a large-scale ontology graph. However, due to space/computing limitations, an experimental subset ($\approx 500,000$ topics) was extracted.

B. Data Extraction

The process of Freebase data extraction was performed over two separate stages. Firstly, the schema Type objects were previously extracted using the stepwise procedures described in [29]. The total number of extracted Freebase types reached 2,120. Secondly, the Topic instances Freebase data were retrieved from downloadable data dumps, publicly

available at [32]. The data dump is archived as single huge tab-separated file, around 250 GB.

C. Structure of the Ontology Graph

Freebase was engineered around a thoroughly structured schema. The schema includes all types of topics that can be found in Freebase data. A “Type” can correspond to a physical or conceptual entity, such as “Person”, “Country” or “Book”. On the other hand, a “Topic” represents one or more instances of those schema types, such as “Albert Einstein”, “France”, or “A Tale of Two Cities”. Freebase characterises the mapping between schema types and topic instances using the so-called “Included-Type” relationships.

In order to build the ontology graph, schema types and topic instances of Freebase were jointly coupled. The schema types and topics were incorporated within a single large graph as nodes. The graph nodes were banded together through edges constructed as “Subclass of” relationships. The edges corresponded to the original “Included-Type” relationships. Specifically, the ontology graph consisted of 500,000 Topic-nodes, 2,120 Type-nodes, and 2,223,030 edges. A simple illustrative example of the ontology graph structure is sketched in Figure 2.

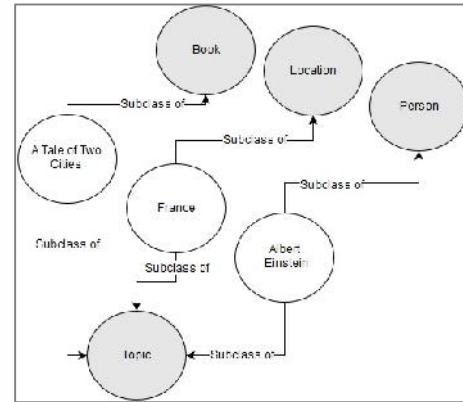


Figure 2. An example of how data is structured within the ontology graph. The graph depicts how schema types and topic instances are linked by “Subclass of” relationships, representing graph edges. (Schema types are gray-shaded, and topic instances with white background)

D. Storage of Ontology Graph

As before-mentioned in section3, Neo4j was decided as the graph database platform. The Neo4j storage model is based on the property graph data model as illustrated in Figure 3. The property graph contains connected nodes that can be tagged with labels and hold related properties in the form of key-value pairs. Relationships provide directed, named connections between two nodes. Similar to nodes, relationships can also have any properties. The ontology graph was loaded into database by importing CSV files that contained graph nodes and edges separately [33]. Furthermore, indexes were created over properties to be used within queries such as “Topic Name” and “Type Name”.

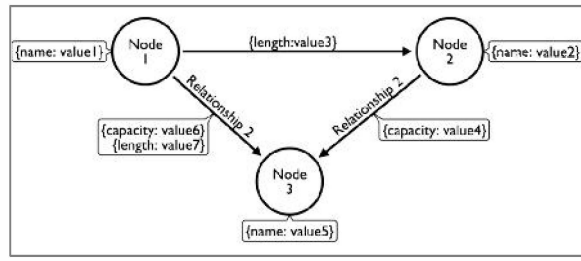


Figure 3. Neo4j's Property Graph Data Model [31]

E. Ontology Query Experiments

Ontology query experiments were conducted using Neo4j's query language, Cypher. The query experiments were conducted on an i5 machine with 2.27 GHz, and 4G RAM running on Windows 7.

We defined 4 query scenarios to be run over the ontology graph, which eventually consisted of 502,120 nodes, and 2,223,030 edges. The complexity of defined queries ranged from simple exact matching to wildcard, or predicate-driven queries. The query response time was recorded by averaging of 5 experiments for each query separately. Table III presents query descriptions, Cypher code, and average query response time.

Table III Description of query scenarios and response time

Query Type	Query Description	Cypher Code	Average Query Response Time (sec)
Exact Match Query	Return all Topics having the exact name of "Albert Einstein"	Match (n:Topic) Using Index n:Topic(TopicName) Where n.TopicName='Albert Einstein' Return n	≈ 0.5
Wildcard Query	Return all Topics having the name starting with "Albert Einstein"	Match (n:Topic) where n.TopicName =~ "Albert Einstein.*" Return n	≈ 18
Predicate-based Query	Return 100 Topics being subclass of the "Book" Type	Match (n:Topic)-[:Subclass_of]->(m:Type) Where m.TypeName='Book' Return n,m limit 100	≈ 2.7
	Return all Topics being subclass of the "Book" Type and having the name of "Albert Einstein"	Match (n:Topic)-[:Subclass_of]->(m:Type) Where n.TopicName='Albert Einstein' and m.TypeName='Book' Return n,m	≈ 1

V. CONCLUSIONS

The paper considers a graph database-oriented approach for realising efficient storage and query of large-scale ontologies, in particular. We develop our view based on findings stemming from a systematic literature review. Obviously, it can be interpreted that there is an evident lack of embracing graph-based models, while other poorly scalable models, such as the relational model, are intensively used. Furthermore, large-scale ontologies are slightly endorsed.

The proposed approach employs Neo4j as the graph database platform. Neo4j was selected for having powerful graph modeling capabilities, graph-driven query language, and scalability. The validity of our approach are verified based on an empirical case study that uses a subset of the huge data of Freebase. The Freebase subset is utilised to construct a large-scale ontology graph consisting of more than 500K nodes, and 2M edges. Diverse sets of queries were efficiently executed over the large ontology graph. The approach is claimed to effectively provide both efficiency and scalability for the purpose of storing and querying ontologies.

REFERENCES

- [1] Angles, Renzo, and Claudio Gutierrez. "Survey of graph database models." *ACM Computing Surveys (CSUR)* 40, no. 1 (2008): 1.
- [2] Silberschatz, Avi, Henry F. Korth, and S. Sudarshan. "Data models." *ACM Computing Surveys (CSUR)* 28, no. 1 (1996): 105-108.
- [3] Paredaens, Jan, Peter Peelman, and Letizia Tanca. "G-Log: A graph-based query language." *Knowledge and Data Engineering, IEEE Transactions on* 7, no. 3 (1995): 436-453.
- [4] Barmpis, Konstantinos, and D. Kolovos. "Evaluation of contemporary graph databases for efficient persistence of large-scale models." *Journal of Object Technology* (to appear, 2014) (2014).
- [5] http://neo4j.com/developer/graph-database/#_what_is_neo4j, retrieved on 19/09/2015.
- [6] <http://neo4j.com/developer/graph-db-vs-rdbms/>
- [7] Dominguez-Sal, David, P. Urbón-Bayes, Aleix Giménez-Vañó, Sergio Gómez-Villamor, Norbert Martínez-Bazan, and Josep-Lluís Larriba-Pey. "Survey of graph database performance on the hpc scalable graph analysis benchmark." In *Web-Age Information Management*, pp. 37-48. Springer Berlin Heidelberg, 2010.
- [8] Angles, Renzo. "A comparison of current graph database models." In *Data Engineering Workshops (ICDEW)*, 2012 IEEE 28th International Conference on, pp. 171-177. IEEE, 2012.
- [9] Booth, Andrew, Diana Papaioannou, and Anthea Sutton. *Systematic approaches to a successful literature review*. Sage, 2011.
- [10] Dehainsala, Hondjack, Guy Pierra, and Ladjel Bellatreche. "Ontodb: An ontology-based database for data intensive applications." In *Advances in Databases: Concepts, Systems and Applications*, pp. 497-508. Springer Berlin Heidelberg, 2007.
- [11] Harris, Stephen, and Nicholas Gibbins. "3store: Efficient bulk RDF storage." (2003): 1-15.
- [12] Agrawal, Rakesh, Amit Somani, and Yirong Xu. "Storage and querying of e-commerce data." In *VLDB*, vol. 1, pp. 149-158. 2001.
- [13] Broekstra, Jeen, Arjohn Kampman, and Frank Van Harmelen. "Sesame: A generic architecture for storing and querying rdf

- and rdf schema." In *The Semantic Web—ISWC 2002*, pp. 54-68. Springer Berlin Heidelberg, 2002.
- [14] Zhou, Zhou, and Xing Yongkang. "A study on ontology storage based on relational database." In *Conference Anthology*, IEEE, pp. 1-5. IEEE, 2013.
- [15] Barranco, Carlos D., Jesus R. Campaña, Juan M. Medina, and Olga Pons. "On storing ontologies including fuzzy datatypes in relational databases." In *Fuzzy Systems Conference*, 2007. FUZZ-IEEE 2007. IEEE International, pp. 1-6. IEEE, 2007.
- [16] Zuling, Kang, and Wang Hongbing. "Implementation and Application of Ontology Databases with User-Defined Rules (UDR) Supported." In *Semantics, Knowledge and Grid*, 2005. SKG'05. First International Conference on, pp. 82-82. IEEE, 2005.
- [17] de Souza Baptista, Cláudio. "OntoEditor: a Web tool for manipulating ontologies stored in database servers." In *Database and Expert Systems Applications*, 2004. Proceedings. 15th International Workshop on, pp. 151-155. IEEE, 2004.
- [18] Khalid, Adnan, Syed Adnan Hussain Shah, and Muhammad Abdul Qadir. "OntRel: An Ontology Indexer to store OWL-DL Ontologies and its Instances." In *Soft Computing and Pattern Recognition*, 2009. SOCPAR'09. International Conference of, pp. 478-483. IEEE, 2009.
- [19] ALAmri, Abdullah. "The relational database layout to store ontology knowledge base." In *Information Retrieval & Knowledge Management (CAMP)*, 2012 International Conference on, pp. 74-81. IEEE, 2012.
- [20] Al-Jadir, Lina, Christine Parent, and Stefano Spaccapietra. "Reasoning with large ontologies stored in relational databases: The OntoMinD approach." *Data & Knowledge Engineering* 69, no. 11 (2010): 1158-1180.
- [21] Ma, Li, Zhong Su, Yue Pan, Li Zhang, and Tao Liu. "RStar: an RDF storage and query system for enterprise resource management." In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, pp. 484-491. ACM, 2004.
- [22] Harrison, Robert, and Christine W. Chan. "Distributed ontology management system." In *Electrical and Computer Engineering*, 2005. Canadian Conference on, pp. 661-664. IEEE, 2005.
- [23] Okure, Akanimo Samuel, and Jeff Z. Pan. "Querying EL Ontology Stream with C-SPARQL." In *Advanced Computer Science Applications and Technologies (ACSAT)*, 2013 International Conference on, pp. 337-340. IEEE, 2013.
- [24] Abadi, Daniel J., Adam Marcus, Samuel R. Madden, and Kate Hollenbach. "Scalable semantic web data management using vertical partitioning." In *Proceedings of the 33rd international conference on Very large data bases*, pp. 411-422. VLDB Endowment, 2007.
- [25] Zhang, Fu, Z. M. Ma, and Weijun Li. "Storing OWL ontologies in object-oriented databases." *Knowledge-Based Systems* 76 (2015): 240-255.
- [26] Zou, Lei, Jinghui Mo, Lei Chen, M. Tamer Özsu, and Dongyan Zhao. "gStore: answering SPARQL queries via subgraph matching." *Proceedings of the VLDB Endowment* 4, no. 8 (2011): 482-493.
- [27] Angles, Renzo, and Claudio Gutierrez. "Querying RDF data from a graph database perspective." In *The Semantic Web: Research and Applications*, pp. 346-360. Springer Berlin Heidelberg, 2005.
- [28] Arrison, Thomas, and Scott Weidman, eds. *Steps Toward Large-Scale Data Integration in the Sciences: Summary of a Workshop*. National Academies Press, 2010.
- [29] Elbattah, Mahmoud, Mohamed Roshdy, Mostafa Aref, and Abdel-Badeh Salem. "GRAPH-DRIVEN ANALYSIS AND VISUALISATION OF FREEBASE SCHEMA AS A DIRECTED WEIGHTED GRAPH." *European Journal of Computer Science and Information Technology* 2, no. 4 (2014): 40-48.
- [30] Gruber, Thomas R. "A translation approach to portable ontology specifications." *Knowledge acquisition* 5, no. 2 (1993): 199-220.
- [31] Van Bruggen, Rik. *Learning Neo4j*. Packt Publishing Ltd, 2014.
- [32] <https://developers.google.com/freebase/data>
- [33] <http://neo4j.com/docs/stable/cypherdoc-importing-csv-files-with-cypher.htm>